

UNIVERSITY OF THE ARMED FORCES - ESPE

Computer Science Department

Object Oriented Programming

36863

Homework05 - UMLReading

LASCANO, JORGE EDISON

October 14, 2026

Object Orientation - Conceptual Modeling

Conceptual Modeling

Focuses on capturing key domain ideas and concepts early and continously throught the software development lifecycle to support reasoning and communication

- UML
  Unified Modeling language is a collection of conceptual modeling languages
  - Use case diagrams
  - Class diagrams
  - Interaction diagrams
  - State charts
- Loosely Coupled Layers
  Recommends organizing software into loosely coupled layers, isolating concerns so that changes in one layer have minimal ripple effects across the entire system
- Preference for Aggregation over Inheritance
  Advises favoring composition and agregation over class inherence to reduce rigid hierarchical coupling and avoid fragile base class problems.

Interface or Abstraction: Highlights the significance, promote design about abstractions and interface to manage complexity and decouple client code from concrete implementation

Introduction to the Unified Modeling Language

UML Overview and History
Trance the evolution of OO concepts from the 1960s to the "Method wars" in the 1990s and the creation of UML as a unified modeling language.

Core Building Blocks: Clasifies UML elements int structural behivor, organizational and annotationland modeling elements.

The Modeling Perspectives:

Details how UML diagrams can be interpreted through three distinct lenses:

- Analyst Perspective:
  Understanding the real-world main concepts.
- Specification Perspective:
  Defining software interfaces and abstractions without implementation details.
- Implementation Perspective:
  Mapping directly to concrete source code classes and components.

Diagram Types and Views:

Categorize structural diagrams and behivor interaction diagrams, advicing developers to select meaningful views and abstraction levels that support core engineering activities.

Use Cases

Actors:
They are external entities, system, or user roles that interact with the system to accomplish goals

Use Cases vs. System Interactions

Explain that uses cases represents user goals independent of specific theenological steps or tehcnical UI workflows

- Use-Case Relationships: Explore keys relationships between uses cases

«include»: Mandatory execution of a reusable sub-goal

«extend»: Optional or extenditional to a base use case

- Generalitation: Specialized variations of a broden goal

Modeling Guidelines: Offer practical tips for discovering acters naming use cases clearly around business goals, and using diagrams to comunicate user-visible system scope.

Objects, Classes, and Basic Class Diagrams

- Objects vs Classes: Defines objects as basic system entities and classes as structural blueprints
- Class Syntax and Notation: Explains standar UML class box compartments: Name, Attributes, Operations, Responsabilities and custom constraints
- Syntax Specifications: Details from UML syntax rules for attribute visibility, data types, multiplicity and operations parameters.
- Relationship: Examines fundamental relationships
  - Generalitation
  - Associations: direct links between clases, including navigation rules
  - Agregation
  - Dependence

UML Quick Reference Guide

Analysis Best Practices: Is important to focusing on real-world problem domain modeling

Use Case Diagrams: Cheat sheet for class compartments, visibilities (+, -, #), multi-class associations, multiplicities and aggregation / composition notation

Behavioral and Dynamic Diagrams

Activities Diagrams: Proces flows, actions, desicions n nodes, and forks

Interaction Diagrams: Communication and sequence diagram showing message exchanges and synchronous calls over lifelines.

State Charts: State, transitions, triggers, and lifecycles of complex system objects.